



ChatGPT 5.1

Share

Add people



2.1 — Common regex functions used in NLP (detailed)

What: Regex functions in Python's `re` module let you search, extract, replace, and split text using patterns.

Key functions, what they do & simple examples

1. `re.compile(pattern)`

- Compiles a regex into a pattern object (faster if reused).
- Example:

python

Copy code

```
import re
p = re.compile(r'\d+')
p.findall("Room 24 and 7")
# -> ['24', '7']
```

2. `re.match(pattern, string)`

- Attempts match **only at the start** of the string.
- Use for format validation (e.g., string must start with a date).
- Example:

python

Copy code

```
re.match(r'Hello', "Hello world") # match
re.match(r'world', "Hello world") # None
```

3. `re.search(pattern, string)`

- Finds the **first occurrence anywhere** in the string.
- Example:

python

Copy code

```
re.search(r'\d+', "Age: 30") # finds '30'
```

4. `re.findall(pattern, string)`

- Returns list of **all non-overlapping matches** (strings).
- Example:



python

Copy code



Ask anything



ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#).



4. `re.findall(pattern, string)`

- Returns list of **all non-overlapping** matches (strings).
- Example:

python

 Copy code

```
re.findall(r'\b\w+\b', "I love NLP") # -> ['I', 'love', 'NLP']
```

5. `re.finditer(pattern, string)`

- Returns iterator of match objects — use when you need spans (start/end).
- Example:

python

 Copy code

```
for m in re.finditer(r'\d+', "12 and 34"):
    print(m.group(), m.start(), m.end())
```

6. `re.sub(pattern, repl, string)`

- Replace matches with `repl`. Used for cleaning (remove URLs, digits).
- Example:

python

 Copy code

```
re.sub(r'https?://\S+', '<URL>', text)
```

7. `re.subn(pattern, repl, string)`

- Same as `sub` but returns `(new_string, count)` so you know how many replacements were made.

8. `re.split(pattern, string)`

- Splits text using the regex delimiter (better than simple `split()` for punctuation).
- Example: `re.split(r'\W+', text)` splits on non-word chars.

NLP uses: tokenization, cleaning tweets (remove mentions/URLs), extract dates/numbers, NER pre-processing, building features.

Exam tip: mention examples (`findall` vs `finditer` differences, `sub` for cleaning) and show one short code snippet.



2.2 — Meta characters: Big brackets, caret (cap ^), backslash (\) (detailed)

What: Meta-characters are special symbols that change meaning of regex (not literal characters).

Important meta-characters with purpose & examples

1. Square brackets `[ ... ]` — character classes

- `[abc]` matches `a` or `b` or `c`.
- Ranges: `[a-z]`, `[0-9]`.
- Negation inside class: `[^0-9]` means any char except digits.
- Example: `re.findall(r'[A-Za-z]+', text)` → extract words.

2. Curly brackets `{m,n}` (big brackets referred)

- Quantifiers specifying counts: `a{3}` (exactly 3 `a`), `\d{2,4}` (2 to 4 digits), `\w{1,}` (`>=1`).
- Useful for strict formats like `\d{4}-\d{2}-\d{2}` (YYYY-MM-DD).

3. Caret `^` (cap) — two uses

- Anchor at start: `^Hello` matches only if string starts with "Hello".
- Negation in class: `[^a-z]` inside `[]` means not a-z.

4. Backslash `\` — escape & predefined classes

- Escape special characters: `\.` matches literal dot, `\\` matches backslash.
- Predefined sequences:
 - `\d` digit = `[0-9]`
 - `\D` non-digit
 - `\w` word char = `[A-Za-z0-9_]`
 - `\W` non-word
 - `\s` whitespace (space, tab, newline)
 - `\S` non-whitespace
- Example: `\b` word boundary (useful to match whole words).

Common pitfalls: forgetting to escape special characters (e.g., `.` matches any char unless you write `\.`). Using `^` vs `\b` incorrectly.

Exam tip: give a couple of examples (class + anchor) and a short regex for email validation (simple): `r'^\w+@\w+\.\w{2,3}$'` and mention it's simplified.

2.3 – Squared brackets, special sequences, asterisk (*), plus (+), question mark (?), curly brackets {} (detailed)

This item groups quantifiers and special sequences – core for controlling repetition.

Quantifiers (what they match)

1. Asterisk `*` – zero or more

- `a*` matches `''`, `a`, `aa`, ...
- Example: `colou*r` matches `color` and `colour`.

2. Plus `+` – one or more

- `\w+` matches one or more word characters (good for words).

3. Question mark `?` – zero or one (optional) / or make quantifier lazy

- `colou?r` matches `color` or `colour`.
- Also used for non-greedy: `.*?` takes shortest match.

4. Curly braces `{m}`, `{m,n}` – exact or range counts

- `\d{3}` exactly three digits; `\d{2,4}` between 2 and 4 digits.

Special sequences (predefined classes & anchors)

- `\d`, `\w`, `\s` etc. (described earlier)
- `\b` word boundary – matches boundary between word and non-word (e.g., `\bcat\b` matches `cat` as a separate word).
- `^` (start), `$` (end), `\A`, `\Z` are anchors.

Greedy vs Lazy quantifiers

- Greedy (default): `.*` tries to match as much as possible.
- Lazy: `.*?` matches as little as possible. Important when parsing HTML-like text to avoid grabbing too much.

Examples for NLP

- Extract hashtags: `r'#\w+'`
- Extract mentions: `r'@\w+'`
- Match URLs (simplified): `r'https?:\/\/\S+'`
- Tokenize words including apostrophes: `r"[A-Za-z]+'[A-Za-z]+'"` or `r'\w+(?:'\w+)?'`



Exam tip: explain quantifiers and show one example where greedy vs lazy changes result

2.4 — Understanding Pattern Objects: `match()` vs `search()`, `finditer()`, logical OR `|`, beginning/end patterns, parenthesis (detailed)

Pattern objects: created using `re.compile(pattern)` — faster, allows reuse and method calls `.match()`, `.search()`, `.finditer()`.

`match()` vs `search()`

- `match()` checks only at the beginning; `search()` looks anywhere.

Example: pattern `\d+`, string `"ID: 123"`.

- `re.match` → None (not at start)
- `re.search` → finds `'123'`.

`finditer()`

- Returns match objects (with `.group()`, `.start()`, `.end()`) for all matches — preferred when you need index spans to highlight entities in text.

Logical OR `|`

- Allows matching one of multiple alternatives.

Example: `cat|dog|bird` matches any of these.

Use parentheses to control scope: `(?:Mr|Mrs)\. [A-Z]\w+` or `(Mr|Mrs)\. ([A-Z]\w+)` to capture title and name.

Beginning and end patterns (anchors)

- `^` anchors start of string, `$` anchors end.

Useful for full-string validation: `r'^\d{10}$'` validates a 10-digit phone number.

Parentheses ⇒ groups & capture

- `(pattern)` captures matched substring for extraction; groups are numbered.
- `(?:pattern)` non-capturing group (used to group without capturing).
- Useful for reordering in replacement: `re.sub(r'(\w+)\s+(\w+)', r'\2 \1', 'first last')` → swaps words.

Examples for NLP

- Extract date components: `r'(\d{4})-(\d{2})-(\d{2})'` → groups: year, month, day.
- Titles + names: `r'(Dr|Mr|Ms)\.?\s+([A-Z][a-z]+)'` – captures title and last name.

Exam tip: mention compiled pattern advantage, show `finditer` usage for spans in NER, and a small example of `sub` with groups (`\1`, `\2`) in replacement.

2.5 – String Modification: `split()`, `sub()`, `subn()` (detailed)

These are the functions you use to change text – crucial in preprocessing.

1. `re.split(pattern, string, maxsplit=0)`

- Splits string by regex delimiter (not literal text).
- Example: `re.split(r'\W+', "Hello, world!")` → `['Hello', 'world', '']` (note possible empty tokens at ends; use filter).
- Use for sentence splitting (`((?<=[.!?])\s+)`) or word tokenization including contractions.

2. `re.sub(pattern, repl, string, count=0)`

- Replace all (or `count` times) occurrences of `pattern` with `repl`.
- `repl` can be a string or a function (callable) that receives a match object – powerful for context-sensitive replacement.

Example replacing URLs:

python

 Copy code

```
text = re.sub(r'https?://\S+', '<URL>', text)
```

- Using groups in replacement: `re.sub(r'(\d{3})-(\d{4})', r'(\1)\2', phone)`

3. `re.subn(pattern, repl, string)`

- Returns `(new_string, n_subs)` so you can log how many replacements occurred. Useful in preprocessing reports.

3. `re.subn(pattern, repl, string)`

- Returns `(new_string, n_subs)` so you can log how many replacements occurred. Useful in preprocessing reports.

Practical NLP examples

- **Remove punctuation:** `re.sub(r'[\^\w\s]', '', text)`
- **Normalize whitespace:** `re.sub(r'\s+', ' ', text).strip()`
- **Expand contractions (callable example):**

python

 Copy code

```
contractions = {"don't": "do not", "I'm": "I am"}
def expand(m):
    return contractions.get(m.group(0).lower(), m.group(0))
text = re.sub("|".join(map(re.escape, contractions.keys())), expand
```

- **Mask sensitive info:** `re.sub(r'\b\d{16}\b', 'CARD', text)` (mask credit card numbers).

Edge cases / pitfalls

- Replacing before tokenizing might break offsets — if you need spans later (NER), keep a mapping or replace conservatively.
- Order of `sub` operations matters (e.g., remove HTML tags before removing punctuation).

Exam tip: show `sub` with callable or group backreferencing, mention `subn` for counts, and explain why regex-based `split` is better than simple `split()` for NLP.

Short model answer (1–2 lines) to write at top of exam answer

“Regular expressions are pattern-based rules (using meta-characters, quantifiers, anchors and groups) that enable powerful text searching, extraction and transformation. Python’s `re` module provides `match`, `search`, `findall`, `finditer`, `split`, `sub` and `compile` functions used extensively in NLP preprocessing and feature extraction.”

★ 1. WORDS — 7 MARK ANSWER

Definition:

A *word* is the smallest meaningful linguistic unit that can stand alone and convey meaning. In traditional linguistics, words are built from morphemes (roots, prefixes, suffixes) and are the basis of human language expression.

Explanation:

In NLP, words serve as the basic building blocks for text processing. However, computers cannot understand text directly, so words must be processed, normalized, and converted into numerical form. Word boundaries usually depend on spaces and punctuation, but this varies across languages (e.g., Chinese has no spaces).

Characteristics of words:

- Can represent concepts (love, computer)
- Can represent actions (run, jump)
- Can represent entities (India, Amazon)
- Can be ambiguous ("bank" = river bank or financial institution)

Examples:

Sentence: "NLP is amazing!"

Words: NLP, is, amazing

Importance in NLP:

- Used for tokenization
- Basis for Bag-of-Words, TF-IDF, and word embeddings
- Used in text classification, translation, and search systems

Applications:

- Document indexing
- Keyword extraction
- Spelling correction
- Word-level embeddings like Word2Vec, GloVe

Conclusion:

Words are fundamental linguistic units and  essential for NLP systems, serving as the foundation for representation, analysis, and modelling.

★ 2. TOKENS — 7 MARK ANSWER

Definition:

A *token* is a symbol or unit of text generated during tokenization. A token may be a word, subword, punctuation symbol, or even a character.

Explanation:

While words are linguistic units, *tokens are computational units* that NLP models operate on. Tokenization divides text into tokens according to rules or statistical patterns.

Types of tokens:

- Word tokens: "I", "love", "NLP"
- Subword tokens: "un", "break", "able"
- Character tokens: "a", "m", "z"
- Punctuation tokens: "!", "."

Examples:

Sentence: "I'm learning NLP!"

Tokens: ["I", "'m", "learning", "NLP", "!"]

Importance:

- Standardizes input for NLP algorithms
- Required for embeddings, vocabulary building, and model inputs
- Helps break complex text into manageable pieces

Applications:

- Language modeling
- Named Entity Recognition
- Machine translation
- Text classification

Conclusion:

Tokens serve as the interface between raw text and machine learning models, making them essential for every NLP pipeline.

★ 3. COUNTING WORDS – 7 MARK ANSWER

Definition:

Counting words refers to computing the frequency of each word/token in a text or corpus, often using a frequency distribution.

Purpose:

Used to analyze text, extract important terms, and build features for machine learning models.

How word counting works:

1. Preprocess text (lowercase, clean punctuation)
2. Tokenize into words
3. Count occurrences using methods like:
 - Python `Counter`
 - NLTK's `FreqDist`
 - Pandas group operations

Example:

Text: "NLP is fun. NLP is powerful."

Counts:

- NLP: 2
- is: 2
- fun: 1
- powerful: 1

Importance:

- Foundation for Bag-of-Words (BoW)
- Useful for TF-IDF computation
- Helps extract keywords and understand vocabulary distribution
- Useful in sentiment classification and topic modeling

Applications:

- Text mining
- Document classification
- Summarization
- Information retrieval



Applications:

- Text mining
- Document classification
- Summarization
- Information retrieval

Limitations:

- Ignores word order (NLP $\neq$ LPN)
- Frequency does not always mean importance

Conclusion:

Word counting is a basic yet essential step in NLP that helps quantify text and extract meaningful patterns for further processing.

★ 4. VOCABULARY — 7 MARK ANSWER

Definition:

A *vocabulary* is the set of all unique tokens present in a corpus. Each token is usually mapped to a unique index for use in machine learning.

Formation of vocabulary:

1. Tokenize all documents
2. Collect unique words
3. Assign Integer IDs (e.g., NLP → 5)
4. Add special tokens:
 - `<PAD>` for padding
 - `<UNK>` for unknown words
 - `<START>` and `<END>` for sequence models

Example vocabulary:

Text: "NLP is fun"

Vocabulary = {NLP:1, is:2, fun:3}

Importance:

- Defines the feature space for NLP models
- Used for one-hot encoding, Bag-of-Words, TF-IDF, embeddings
- Helps limit model complexity by controlling vocabulary size

Challenges:

- Large vocabulary increases memory usage
- Rare words increase sparsity
- OOV (Out-of-Vocabulary) problem

Applications:

- Sentiment classification
- Machine translation
- Topic modeling
- Speech recognition

Conclusion:

Vocabulary provides the mapping from text  numbers and forms the backbone of NLP feature representation.

★ 5. CORPUS – 7 MARK ANSWER

Definition:

A *corpus* is a large structured collection of text documents used for training and evaluating NLP models.

Characteristics:

- May contain millions of sentences
- Can be domain-specific (medical, legal) or general-purpose
- Must be cleaned, tokenized, and standardized before use

Examples of corpora:

- Brown Corpus
- Wikipedia dump
- IMDB movie reviews
- Yelp reviews
- Common Crawl Web Corpus

Importance:

- Provides real-world language data
- Helps train statistical NLP models
- Essential for creating embeddings and language models
- Enables evaluation of NLP tasks (NER, POS-tagging, parsing)

Applications:

- Training word embeddings (Word2Vec, GloVe)
- Machine translation corpora like EuroParl
- Text classification datasets
- Building probabilistic language models

Challenges:

- Data cleaning
- Bias in text sources
- Large size requires computational power

Conclusion:

A corpus is the fundamental dataset that enables statistical learning and benchmarking in NLP.

★ 6. TOKENIZATION IN SPACY – 7 MARK ANSWER

Definition:

Tokenization in spaCy refers to splitting text into meaningful tokens using spaCy's high-performance rule-based + statistical tokenizer.

How spaCy tokenizes:

spaCy applies a pipeline of processing steps:

1. Token matching rules (prefix, suffix, and infix patterns)
2. Detects punctuation, contractions, emojis
3. Splits text into a `Doc` object containing individual token objects
4. Assigns lexical attributes (`is_alpha`, `is_digit`, `is_punct`)

Example:

python

 Copy code

```
import spacy
nlp = spacy.load("en_core_web_sm")
doc = nlp("I'm learning NLP!")
[t.text for t in doc]
```

Output:

```
['I', "'m", 'learning', 'NLP', '!']
```

Advantages of spaCy tokenization:

- Extremely fast and efficient
- Handles edge cases (e.g., "U.S.A.", "don't" → do + n't)
- Supports custom tokenization rules
- Integrates with POS tagging, dependency parsing, NER

Applications:

- Text preprocessing pipelines
- Chatbot development
- Feature extraction for machine learning
- Training custom NLP models

★ Sentiment Classification Using Yelp Dataset – 7 Mark Detailed Answer

Sentiment classification aims to automatically determine whether a review expresses **positive** or **negative** sentiment. Using the Yelp review dataset, the entire NLP pipeline involves **data preparation, vocabulary building, encoding, model training, and evaluation.**

◆ 1. Data Preparation using NumPy and Pandas

Yelp reviews are usually downloaded as CSV or JSON files.

- **Pandas** is used to load the dataset:

```
python
```

[Copy code](#)

```
import pandas as pd
df = pd.read_csv("yelp.csv")
```

- Missing values, duplicates, emojis, HTML tags, and irrelevant fields are removed.
- **NumPy** helps convert lists and arrays into efficient numerical formats for model input.

Text Cleaning Steps:

- lowercasing
- removing punctuation
- removing URLs using regex
- expanding contractions ("don't → do not")
- removing digits

These steps ensure consistent text representation.

◆ 2. Tokenization and Counting Words using Counter

After cleaning, the text is **tokenized** into individual words.

python

 Copy code

```
from collections import Counter
word_counts = Counter(tokenized_words)
```

- **Counter** helps determine **frequency** of each token.
- Rare words may be removed; frequent words become part of vocabulary.

◆ 3. Re-adding Tokens to Vocabulary & Handling Special Tokens

A vocabulary must include:

- **<PAD>** – padding for equal length
- **<UNK>** – unknown words
- **<START>**, **<END>** – useful in sequence models

These tokens support variable-length documents and unknown word handling.

If a token is removed during cleaning (rare/low frequency), it is **re-added** explicitly to avoid mismatch during encoding.

◆ 4. Building Vocabulary from DataFrame / Corpus

Two methods can be used:

A) Building vocabulary from DataFrame

- Combine all reviews into one text column.
- Extract unique tokens.
- Map each token to an integer ID.

B) Building vocabulary from full corpus

- Consider entire dataset, not only training data.
- Helps reduce out-of-vocabulary (OOV) issues.

Example:

python

 Copy code

```
vocab = {word:idx for idx, word in enumerate(unique_words)}
```

◆ 5. One-Hot Encoding

Each word in the vocabulary is represented as a vector where:

- length = size of vocabulary
- all values = 0 except the target word = 1

Example: Vocabulary = [good, bad, food, price]

"good" → [1, 0, 0, 0]

Use:

- Useful for simple ML models
- Basis for Bag-of-Words and TF-IDF

◆ 6. Encoding Documents

Each review is converted into a numerical vector.

Two common methods:

A) Bag-of-Words Encoding

Count occurrences of each vocabulary word in the document.

B) Sequence Encoding

Convert token sequence into IDs:

Example:

Sentence = "food was good"

IDs = [12, 45, 78]

Documents are padded using `<PAD>` token to maintain equal length.

◆ 7. Train–Test Split

Dataset is divided into:

- **Training set (70–80%)**
- **Testing set (20–30%)**

Using scikit-learn:

python

 Copy code

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

This helps evaluate model on unseen data.

◆ 8. Feature Computation

Feature computation transforms encoded documents into meaningful inputs.

Common features:

- **Bag-of-Words vectors**
- **TF-IDF scores**
- **n-grams (bigrams, trigrams)**
- **Average word embeddings**

Example TF-IDF:

python

 Copy code

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer()
X = tfidf.fit_transform(df['review'])
```

◆ 9. Training the Sentiment Classifier

Models commonly used:

- Naive Bayes
- Logistic Regression
- SVM
- Deep learning models (LSTM, GRU, Transformers)

Example:

python

 Copy code

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
```

◆ 10. Confusion Matrix and Evaluation

Confusion matrix shows:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Used to compute:

- Accuracy
- Precision
- Recall
- F1-score

Example:

python

 Copy code

```
from sklearn.metrics import confusion_matrix  
cm = confusion_matrix(y_test, y_pred)
```

◆ 11. Analysis

Analysis includes:

- misclassified examples
- performance comparison between models
- effect of stopwords, stemming, vocabulary size
- whether positive/negative words have meaningful weights
- impact of feature choice (TF-IDF performs better than BoW)

★ Language Independent Tokenization – 7 Mark Detailed Answer

Tokenization is the process of splitting text into smaller units (tokens). However, languages differ widely:

- English uses spaces between words
- Chinese, Japanese, Thai have **no spaces**
- Agglutinative languages like Turkish or Malayalam combine many morphemes into long words

To handle all languages, NLP uses **language-independent tokenization methods**: **word**, **character**, and **sub-word** tokenization.

◆ 1. Word Tokenization

Definition:

Splits text into words based on spaces and punctuation.

Example:

"I love NLP" → ["I", "love", "NLP"]

✓ Advantages

- Easy to implement
- Produces meaningful units for many languages
- Works well for English/space-separated languages

✗ Problems with Word Tokenizer

- Fails for languages without spaces (Chinese, Japanese)
- Cannot handle agglutination:
Example: Turkish word → "evlerinizden" (*from your houses*) = many morphemes
- Very large vocabulary (millions of words)
- Out-of-Vocabulary (OOV) problem — unseen words cannot be processed
- Misspellings and spelling variations treated as different words



◆ 2. Character Tokenization

Definition:

Splits text into individual characters.

Example:

"NLP" → ["N", "L", "P"]

✓ Advantages

- Truly language independent; works for **all languages**
- No OOV problem — every character exists in vocabulary
- Good for morphologically rich languages

✗ Drawbacks of Character-Based Tokenizer

- Sequences become very long → slower training
- Loses semantic meaning (characters carry less information than words)
- Models need to learn word boundaries and structure from scratch
- Harder for models to understand long-range dependencies

◆ 3. Sub-word Tokenization

Sub-word tokenization splits text into units **between characters and words**.

Example:

"unbelievable" → ["un", "believe", "able"]

"playing" → ["play", "ing"]

✓ Advantages

- Reduces vocabulary size drastically
- Handles OOV words because unknown words are broken into known subwords
- Works for all languages including morphologically rich languages
- Used by modern NLP models like **BERT, GPT, RoBERTa**

✗ Problems with Sub-word Tokenization

- Requires training on a large corpus
- Sometimes splits words in unnatural ways
Example: "running" → "runn" + "ing"
- Hard to interpret the subword units
- Different tokenizers create different segmentations → re-training needed

★ 4. Byte Pair Encoding (BPE)

(VERY Important — explicitly mentioned in your syllabus)

BPE is one of the most popular **sub-word tokenization algorithms**.

How BPE Works

1. Start with **characters** as initial tokens.
2. Count all **adjacent symbol pairs**.
3. Merge the **most frequent pair** into a new subword token.
4. Repeat merges until vocabulary reaches target size.

Example:

Text: "lower low lowest"

Initial tokens:

l o w e r | l o w | l o w e s t

Frequent pair = "lo" → merge

Next frequent pair = "low" → merge

Resulting subwords:

"low", "er", "est", etc.

✓ Advantages

- Efficient for rare words
- Works across languages
- Reduces OOV problem
- Produces compact vocabulary

Used by **GPT, OpenAI models, RoBERTa, MT systems**

✗ Limitations

- May produce unnatural splits
- Requires preprocessing on entire corpus
- Tokenization is fixed — cannot adapt to new tasks without retraining

★ String Matching and Spelling Correction Using Minimum Edit Distance (7-Mark Answer)

String matching and spelling correction are essential NLP tasks used to compare words, detect misspellings, and suggest the closest correct word. The most widely used method is the **Minimum Edit Distance (MED)**, also known as **Levenshtein Distance**.

◆ 1. Minimum Edit Distance – Definition

Minimum Edit Distance is the **minimum number of operations** required to transform one string into another.

Allowed operations are:

1. **Insertion**
2. **Deletion**
3. **Substitution**

Cost is usually:

- insertion = 1
- deletion = 1
- substitution = 1 (0 if characters match)

Minimum Edit Distance is used in:

- Spelling correction
- String similarity
- Machine translation
- Speech recognition

◆ 2. Dynamic Programming Approach (Core Logic)

The algorithm uses **dynamic programming** to compute the edit distance efficiently.

Steps:

1. Create a matrix **D** of size $(m+1) \times (n+1)$
 - `m = len(source word)`
 - `n = len(target word)`
2. Initialize first row and first column:
 - `D[i][0] = i` (cost of deleting *i* characters)
 - `D[0][j] = j` (cost of inserting *j* characters)
3. Fill the DP table using this formula:

bash

 Copy code

```
if source[i-1] == target[j-1]:
    cost = 0
else:
    cost = 1

D[i][j] = min(
    D[i-1][j] + 1,      # deletion
    D[i][j-1] + 1,      # insertion
    D[i-1][j-1] + cost  # substitution
)
```

4. Final distance = `D[m][n]`

◆ 3. Table Filling Example (VERY IMPORTANT FOR EXAMS)

Find minimum edit distance between:

"cat" → "cut"

Matrix initialization:

	∅	c	u	t
∅	0	1	2	3
c	1	0	1	2
a	2	1	1	2
t	3	2	2	1

Explanation:

- "c" → "c": 0 cost
- "a" → "u": substitution cost 1
- "t" → "t": 0 cost

Final answer:

➡ Minimum Edit Distance = 1

(Only "a" → "u" substitution)

◆ 4. Spell Correction Using Edit Distance

Spelling correction chooses the dictionary word with the **minimum edit distance** from the misspelled word.

How it works:

1. Given misspelled word: "recieve"
2. Compute MED with possible candidates:
 - receive
 - review
 - relieve
3. The candidate with **lowest distance** is selected.

Extensions:

- Weighting operations (substitution cost for similar sounds is low)
- Damerau-Levenshtein distance (includes transposition swapping of letters)

◆ 5. Why Dynamic Programming?

Dynamic Programming is used because:

- Edit distance has overlapping subproblems
- DP reduces computation from exponential $\rightarrow O(m \times n)$
- Table filling ensures optimal solution

Thus, the algorithm is accurate and efficient.

★ Conclusion

Minimum Edit Distance provides a systematic way to measure similarity between strings using dynamic programming. It is the foundation of spell checkers, auto-correct systems, search engines, and many NLP applications. Table filling ensures efficient calculation, making MED one of the most important algorithms in text processing and spelling correction.

